

Teamwork IV: Software Requirements specification

Team 1	Student number	Student name
	e2401834	Karyna Misnik

1. What are the main responsibilities of the customer and the software engineer during requirements engineering? Discuss why both sides need to participate actively.

Customer	Software engineer
Explain business needs and goals	Understand and analyze customer needs
Describe required features	Identify functional and non-functional requirements
Provide domain knowledge	Suggest technical solutions and alternatives
Clarify priorities	Check feasibility, cost, and constraints
Review and validate requirements	Document requirements clearly
Give feedback and approve requirements	Identify conflicts, risks, and missing requirements

Both sides are needed because the customer knows what the business needs, while the software engineer knows how those needs can be implemented.

2. Why is communication difficult during requirements analysis? As a team, identify common causes of misunderstanding and suggest ways to reduce them.

Communication is difficult because customers, users, and developers may have different knowledge, terminology, expectations, and interpretations of what the system should do.

Cause of misunderstanding	How to reduce it
Different technical knowledge	Use simple, clear language
Ambiguous requirements	Write specific and measurable requirements
Different terminology	Define important terms and acronyms
Different expectations	Discuss requirements together and confirm understanding
Requirements change	Document and communicate changes
Poor communication	Hold regular meetings and discussions
Missing information	Ask questions and clarify requirements
Different interpretations	Use examples, diagrams, and prototypes

3. Requirements analysis focuses on what the system should do rather than how it should be implemented. Discuss why this separation is useful during early analysis.

Separating what from how helps the team focus on the user's needs and system requirements without being limited by a specific technology or implementation.

It also makes requirements clearer, more flexible, and easier to change during the early stages.

4. What information should an analyst collect during problem recognition? Create a list of questions that could help a team understand the customer's real problem.

Information to collect during problem recognition

- What is the current problem?
- Who is affected by the problem?
- What are the main goals?
- How is the problem currently handled?
- What difficulties or limitations exist?
- What does the customer expect from the new system?
- What are the main priorities?
- Are there any constraints (time, budget, technology)?
- What would success look like?
- Are there any existing systems or processes that must be considered?

5. Compare a simple question-and-answer interview with the FAST approach. Discuss the strengths of each approach and explain when a team might prefer a collaborative meeting.

Approach	Strengths	When to use
Question-and-answer interview	Simple, structured, easy to prepare and document	When you need specific information from one person
FAST approach	More structured collaboration; helps discover and clarify requirements	When requirements are complex or unclear
Collaborative meeting	Brings customers, users, and developers together; different viewpoints can be discussed immediately	When there are many stakeholders or conflicting requirements

6. Design a FAST meeting for requirements gathering. Decide who should participate, what rules should be followed, how ideas should be recorded, and how the final consensus list should be created.

FAST meeting for requirements gathering

- Participants: Customer/user representatives, software engineers, business analyst, and a facilitator.
- Rules: Everyone can contribute, listen to others, stay focused on requirements, avoid personal criticism, and resolve disagreements through discussion.
- Recording ideas: Write requirements on a shared document/whiteboard and group similar ideas together.
- Creating consensus: Review all requirements together, clarify unclear points, remove duplicates, discuss conflicts, prioritize them, and agree on a final list that everyone accepts.

7. What is the purpose of a mini-specification? Discuss what information should be included in a good mini-specification and how it can help later development activities.

A mini-specification describes in detail how a specific process or function should work. It should include:

- Inputs
- Processing / logic
- Outputs
- Conditions and exceptions

It helps developers understand the requirement clearly and provides a basis for design, coding, and testing later.

8. Compare the three QFD requirement types: normal, expected, and exciting. Create several examples for a software system and explain why each example belongs to its category.

Type	Meaning	Software example	Why?
Normal	User explicitly wants it	Online hotel booking	Customer expects to be able to book a room
Expected	User assumes it will exist	Secure payment	Users expect their payment to be handled securely
Exciting	Not expected, but pleasantly surprising	Automatic room recommendation	Provides extra value that the customer did not specifically request

9. How can prototyping help when customers and developers are not completely sure about the requirements? Discuss its possible benefits for communication, understanding, and requirements refinement.

Prototyping creates an early model of the system that customers and developers can see and discuss.

It helps by:

- Improving communication through something visual.
- Helping customers and developers understand requirements better.
- Revealing missing or incorrect requirements early.
- Allowing requirements to be refined through feedback before full development.

10. How can FAST and QFD be used together during requirements engineering?

Develop a team process that shows how customer needs could be collected, discussed, prioritized, and transformed into software requirements.

FAST + QFD process

1. FAST meeting – Gather customer needs and problems through discussion.
2. Clarify needs – Remove misunderstandings and group similar needs.
3. Prioritize needs – Decide which customer needs are most important.
4. QFD – Convert the prioritized WHATs (customer needs) into HOWs (technical/software requirements).
5. Review – Developers and customers check that the requirements are realistic and complete.
6. Document – Create the final agreed requirements for design and development.